

10-Reflection(反射)

鸣谢：黑马程序员



💡 Tip

Previous Chapter(上一章): 09-Network Communication(网络通信)

- [🔗Markdown](#)
- [🔗PDF](#)
- [🔗HTML](#)

Next Chapter(下一章): 11-Annotation(注解)

- [🔗Markdown](#)
- [🔗PDF](#)
- [🔗HTML](#)

• 10-Reflection(反射)

一、概述

二、操作步骤

1.加载类，获取类的字节码：Class 对象

1.1 获取 Class 对象的3种方式

1.2 代码演示

2.获取类的构造器： `Constructor` 对象

2.1 常用方法

P1 `Class` 类提供的方法

P2 `Constructor` 类提供的方法

2.2 代码演示

2.3 获取构造器的作用：初始化对象并返回

3.获取类的成员变量： `Field` 对象

3.1 常用方法

P1 `Class` 类提供的方法

P2 `Field` 类提供的方法

3.2 代码演示

3.3 获取成员变量的作用：赋值与取值

4.获取类的成员方法： `Method` 对象

4.1 常用方法

P1 `Class` 类提供的方法

P2 `Method` 类提供的方法

4.2 代码演示

4.3 获取成员方法的作用：执行方法

三、作用及应用场景

1.概述

2.案例

2.1 背景

2.2 需求

2.3 实现思路

2.4 实现代码

2.5 测试

一、概述

Module java.base 模块 java.base

Package java.lang.reflect

包 java.lang.reflect

package java.lang.reflect

Provides classes and interfaces for obtaining reflective information about classes and objects. Reflection allows programmatic access to information about the fields, methods, and constructors of loaded classes, and the use of reflected fields, methods, and constructors to operate on their underlying counterparts, within encapsulation and security restrictions.

提供类和接口，用于获取关于类和对象的反射信息。反射允许程序化访问已加载类的字段、方法和构造函数的信息，并使用反射的字段、方法和构造函数在封装和安全限制内操作它们的底层对应物。

Classes in this package, along with java.lang.Class accommodate applications such as debuggers, interpreters, object inspectors, class browsers, and services such as Object Serialization and JavaBeans that need access to either the public members of a target object (based on its runtime class) or the members declared by a given class.

本包中的类以及 java.lang.Class 支持诸如调试器、解释器、对象检查器、类浏览器等应用程序，以及需要访问目标对象（基于其运行时类）的公共成员或给定类声明的成员的的服务，例如对象序列化和 JavaBeans。

二、操作步骤

1. 加载类，获取类的字节码：Class 对象

1.1 获取 Class 对象的3种方式

- `Class c1 = 目标类名.class`
- 调用 `Class` 类提供的静态方法：`public static Class.forName(String package)`
 - `Class c2 = Class.forName(目标类的完整类名, 即包括所在包名)`
- 调用 `Object` 类提供的方法：`public Class getClass()`
 - `Class c3 = 目标类的实例对象.getClass()`

1.2 代码演示

首先在模块下新建一个包 `reflection`，然后创建一个学生类 `Student` 和一个测试类 `Test1`。

- `Student`：

```
1 package reflection;
2
3 import java.util.Objects;
4
5 public class Student {
```

```

6     private String name;
7     private int age;
8     private double height;
9
10    public Student() {
11    }
12
13    public Student(String name, int age, double height) {
14        this.name = name;
15        this.age = age;
16        this.height = height;
17    }
18
19    // 省略getter&setter
20 }

```

- Test1:

```

1  package reflection;
2
3  public class Test1 {
4      public static void main(String[] args) throws Exception {
5          Class c1 = Student.class;
6          System.out.println(c1.getName()); // 完整类名，包括前面的包名
7          System.out.println(c1.getSimpleName()); // 简名: Student
8
9          Class c2 = Class.forName("reflection.Student");
10         if (c2.equals(c1)) {
11             System.out.println("c2 == c1");
12         }
13
14         Student s = new Student();
15         Class c3 = s.getClass();
16         if (c3.equals(c1)) {
17             System.out.println("c3 == c1");
18         }
19     }
20 }

```

- 控制台输出:

```
D:\CS-Softwares\JDK\JDK21\bin\java.exe "-javaagent:D:\CS-!
reflection.Student
Student
c2 == c1
c3 == c1

Process finished with exit code 0
```

2. 获取类的构造器： `Constructor` 对象

2.1 常用方法

P1 `Class` 类提供的方法

序号	方法	说明
01	<code>Constructor<?>[] getConstructors()</code>	获取全部 <code>public</code> 构造器。
02	<code>Constructor<?>[] getDeclaredConstructors()</code>	获取全部构造器。
03	<code>Constructor<?>[] getConstructor(Class<?>... parameterTypes)</code>	获取某个 <code>public</code> 构造器。
04	<code>Constructor<?>[] getDeclaredConstructor(Class<?>... parameterTypes)</code>	获取某个构造器。

P2 Constructor 类提供的方法

序号	方法	说明
01	<code>Object newInstance(Object... initargs)</code>	调用此构造器对象表示的构造器，传入参数，完成对象的初始化并返回。
02	<code>void setAccessible(boolean flag)</code>	当 <code>flag</code> 设置为 <code>true</code> 时，表示禁止编译器检查访问权限，即 暴力反射 。

2.2 代码演示

在包 `reflection` 下继续创建一个猫类 `Cat` 和一个测试类 `Test2`。

- `Cat`：

```
1 package reflection;
2
3 public class Cat {
4     private String name;
5     private int age;
6
7     public Cat() {
8         System.out.println("no-param constructor is executed");
9     }
10
11     private Cat(String name) {
12         this.name = name;
13         System.out.println("private Cat(String name) is executed");
14     }
15
16     private Cat(int age) {
17         System.out.println("private Cat(int age) is executed");
18         this.age = age;
19     }
20
21     public Cat(String name, int age) {
```

```

22         this.name = name;
23         this.age = age;
24         System.out.println("full-params constructor is executed");
25     }
26
27     @Override
28     public String toString() {
29         return "Cat{" +
30             "name='" + name + '\'' +
31             ", age=" + age +
32             '}';
33     }
34
35     // 省略getter&setter
36 }

```

- Test2:

```

1  package reflection;
2
3  import java.lang.reflect.Constructor;
4  import java.util.Arrays;
5
6  public class Test2 {
7      public static void main(String[] args) throws Exception {
8          Class c = Cat.class;
9
10         System.out.println("====All Public
11 Constructors====");
12         Arrays.stream(c.getConstructors()).forEach(constructor -> {
13             System.out.println(constructor.toString() + " ==> " +
14             constructor.getParameterCount() + " params");
15         });
16         System.out.println();
17
18         System.out.println("====All Constructors====");
19
20         Arrays.stream(c.getDeclaredConstructors()).forEach(constructor ->
21         {
22             System.out.println(constructor.toString() + " ==> " +
23             constructor.getParameterCount() + " params");
24         });
25         System.out.println();
26     }
27 }

```

```

22         Constructor noParamConstructor = c.getConstructor();
23         System.out.println("no-param constructor: " +
            noParamConstructor);
24         Constructor fullParamsConstructor =
            c.getDeclaredConstructor(String.class, int.class);
25         System.out.println("full-params constructor: " +
            fullParamsConstructor);
26     }
27 }

```

- 控制台输出:

```

D:\CS-Softwares\JDK\JDK21\bin\java.exe "-javaagent:D:\CS-Softwares\IntelliJ IDEA Ult
=====All Public Constructors=====
public reflection.Cat() ==> 0 params
public reflection.Cat(java.lang.String,int) ==> 2 params

=====All Constructors=====
private reflection.Cat(java.lang.String) ==> 1 params
public reflection.Cat() ==> 0 params
private reflection.Cat(int) ==> 1 params
public reflection.Cat(java.lang.String,int) ==> 2 params

no-param constructor: public reflection.Cat()
full-params constructor: public reflection.Cat(java.lang.String,int)

Process finished with exit code 0

```

2.3 获取构造器的作用：初始化对象并返回

```

1 package reflection;
2
3 import java.lang.reflect.Constructor;
4 import java.util.Arrays;
5
6 public class Test2 {
7     public static void main(String[] args) throws Exception {
8         Class c = Cat.class;
9

```



```

10         Constructor constructor =
c.getDeclaredConstructor(String.class);
11         constructor.setAccessible(true);// 允许访问私有构造器，会破坏封装性
12         Cat cat = (Cat) constructor.newInstance("狸花猫");
13         System.out.println(cat);
14     }
15 }

```

控制台输出：

```

D:\CS-Softwares\JDK\JDK21\bin\java.exe "-javaagent:D:\
private Cat(String name) is executed
Cat{name='狸花猫', age=0}

Process finished with exit code 0

```

3. 获取类的成员变量：Field 对象

3.1 常用方法

P1 Class 类提供的方法

序号	方法	说明
01	Field[] getFields()	获取全部 public 成员变量。
02	Field[] getDeclaredFields()	获取全部成员变量。
03	Field getField(String name)	获取某个 public 成员变量。
04	Field getDeclaredField(String name)	获取某个成员变量。

P2 Field 类提供的方法

序号	方法	说明
01	<code>void set(Object obj, Object value)</code>	赋值。
02	<code>Object get(Object obj)</code>	取值。
03	<code>void setAccessible(boolean flag)</code>	当 <code>flag</code> 设置为 <code>true</code> 时，表示禁止编译器检查访问权限，即暴力反射。

3.2 代码演示

- `Cat` :

```
1 package reflection;
2
3 public class Cat {
4     public static int a;
5     public static final String COUNTRY = "CHINA";
6     private String name;
7     private int age;
8
9     public Cat() {
10         System.out.println("no-param constructor is executed");
11     }
12
13     private Cat(String name) {
14         this.name = name;
15         System.out.println("private Cat(String name) is executed");
16     }
17
18     private Cat(int age) {
19         System.out.println("private Cat(int age) is executed");
20         this.age = age;
21     }
22 }
```

```

22
23     public Cat(String name, int age) {
24         this.name = name;
25         this.age = age;
26         System.out.println("full-params constructor is executed");
27     }
28
29     @Override
30     public String toString() {
31         return "Cat{" +
32             "name='" + name + '\'' +
33             ", age=" + age +
34             "'}";
35     }
36
37     // 省略getter&setter
38 }

```

- Test3:

```

1  package reflection;
2
3  import java.lang.reflect.Field;
4  import java.util.Arrays;
5
6  public class Test3 {
7      public static void main(String[] args) throws Exception {
8          Class c = Cat.class;
9
10         System.out.println("====All Fields====");
11         Arrays.stream(c.getDeclaredFields()).forEach(f -> {
12             System.out.println(f.getName() + " ==> " +
13 f.getType());
14         });
15         System.out.println();
16
17         Field fName = c.getDeclaredField("name");
18         System.out.println(fName.getName() + " ==> " +
19 fName.getType());
20     }
21 }

```

- 控制台输出:

```
D:\CS-Softwares\JDK\JDK21\bin\java.exe "-javaagent:D:\C
=====All Fields=====
a ==> int
COUNTRY ==> class java.lang.String
name ==> class java.lang.String
age ==> int

name ==> class java.lang.String

Process finished with exit code 0
```

3.3 获取成员变量的作用：赋值与取值

```
1 package reflection;
2
3 import java.lang.reflect.Field;
4
5 public class Test3 {
6     public static void main(String[] args) throws Exception {
7         Class c = Cat.class;
8
9         Field fName = c.getDeclaredField("name");
10        System.out.println(fName.getName() + " ==> " +
fName.getType());
11        fName.setAccessible(true);
12        Cat cat = new Cat();
13
14        fName.set(cat, "三花猫");
15        System.out.println("赋值后: " + cat);
16        String name = (String) fName.get(cat); // 强转
17        System.out.println("取值: " + name);
18
19    }
20 }
```

控制台输出：

```
D:\CS-Softwares\JDK\JDK21\bin\java.exe "-javaagent:D:\CS-Softwa
name ==> class java.lang.String
no-param constructor is executed
赋值后: Cat{name='三花猫', age=0}
取值: 三花猫

Process finished with exit code 0
```

4. 获取类的成员方法：Method 对象

4.1 常用方法

P1 Class 类提供的方法

序号	方法	说明
01	<code>Method[] getMethods()</code>	获取全部 <code>public</code> 成员方法。
02	<code>Method[] getDeclaredMethods()</code>	获取全部成员方法。
03	<code>Method getMethod(String name, Class<?>... parameterTypes)</code>	获取某个 <code>public</code> 成员方法。
04	<code>Method getDeclaredMethod(String name, Class<?>... parameterTypes)</code>	获取某个成员方法。

P2 Method 类提供的方法

序号	方法	说明
01	<code>Object invoke(Object obj, Object... args)</code>	执行目标对象的指定方法。
02	<code>void setAccessible(boolean flag)</code>	当 <code>flag</code> 设置为 <code>true</code> 时，表示禁止编译器检查访问权限，即 暴力反射 。

4.2 代码演示

- `Cat` :

```
1 package reflection;
2
3 public class Cat {
4     private String name;
5     private int age;
6
7     public Cat() {
8         System.out.println("no-param constructor is executed");
9     }
10
11     public Cat(String name, int age) {
12         this.name = name;
13         this.age = age;
14         System.out.println("full-params constructor is executed");
15     }
16
17     private void run() {
18         System.out.println("cats run fast");
19     }
20
21     public void eat() {
22         System.out.println("cats like to eat fish");
23     }
24 }
```

```

24
25     private String eat(String foodName) {
26         return "cats like to eat " + foodName + " best";
27     }
28
29     @Override
30     public String toString() {
31         return "Cat{" +
32             "name='" + name + '\'' +
33             ", age=" + age +
34             "'}";
35     }
36
37     // 省略getter&setter
38 }

```

- Test4:

```

1  package reflection;
2
3  import java.lang.reflect.Method;
4  import java.util.Arrays;
5
6  public class Test4 {
7      public static void main(String[] args) throws Exception {
8          Class c = Cat.class;
9          Arrays.stream(c.getDeclaredMethods()).forEach(method -> {
10              System.out.println(method.getName() + "{"
11                  + method.getParameterCount() + " params ; "
12                  + "return type: " + method.getReturnType() +
13              "}");
14          });
15          System.out.println();
16
17          Method eat = c.getDeclaredMethod("eat", String.class);
18          System.out.println(eat.getName() + "{"
19              + eat.getParameterCount() + " params ; "
20              + "return type: " + eat.getReturnType() + "}");
21      }
22  }

```

- 控制台输出:

```
D:\CS-Softwares\JDK\JDK21\bin\java.exe "-javaagent:D:\CS-Soft
getAge{0 params ; return type: int}
eat{1 params ; return type: class java.lang.String}
eat{0 params ; return type: void}
setAge{1 params ; return type: void}
getName{0 params ; return type: class java.lang.String}
run{0 params ; return type: void}
toString{0 params ; return type: class java.lang.String}
setName{1 params ; return type: void}

eat{1 params ; return type: class java.lang.String}

Process finished with exit code 0
```

4.3 获取成员方法的作用：执行方法

```
1 package reflection;
2
3 import java.lang.reflect.Method;
4 import java.util.Arrays;
5
6 public class Test4 {
7     public static void main(String[] args) throws Exception {
8         Class c = Cat.class;
9
10        Method eat = c.getDeclaredMethod("eat", String.class);
11        Cat cat = new Cat();
12        System.out.println(eat.getName() + "{ "
13            + eat.getParameterCount() + " params ; "
14            + "return type: " + eat.getReturnType() + "}");
15        eat.setAccessible(true);
16        System.out.println(eat.invoke(cat, "猫条"));
17    }
18 }
```

控制台输出：


```
D:\CS-Softwares\JDK\JDK21\bin\java.exe "-javaagent:D:\CS-Softwares\JDK\JDK21\bin\java.exe" -no-param constructor is executed
eat{1 params ; return type: class java.lang.String}
cats like to eat 猫条 best

Process finished with exit code 0
```

三、作用及应用场景

1.概述

- **基本作用**：获取一个类的所有成员并进行相关操作。
- 可以破坏类的封装性。
- **重要作用**：适合做Java的框架，当代主流框架都会基于反射设计出一些通用功能。

2.案例

2.1 背景

假设现在有两个类：

- **Student**：

```
1 public class Student {
2     private String name;
3     private int age;
4     private char sex;
5     private double height;
6     private String hobby;
7 }
```

- **Teacher**：

```
1 public class Teacher {  
2     private String name;  
3     private double salary;  
4 }
```

然后在主程序中创建出两个对象：

```
1 new Student("zsh",20,'m',175.0,"soccer");  
2 new Teacher("aj",3000);
```

我们想要把这两个对象的字段名和对应的值，保存到文件中，那么就需要分别为这两个类编写相应的业务逻辑代码。

可是一旦有成百上千个类，创建出成百上千个实例对象，要实现以上需求，如果还是分别为这成百上千个类编写相应的业务逻辑代码，显然重复累赘、效率低下，是不现实的。

2.2 需求

使用反射做一个简易框架，对于任意对象，该框架都可以把该对象的字段名和对应的值，保存到文件中。

2.3 实现思路

1. 定义一个方法，可以接收任意对象；
2. 每收到一个对象，就使用反射获取该对象的 `Class` 对象，然后获取全部成员变量；
3. 遍历全部成员变量，提取成员变量在该对象中的具体值；
4. 将成员变量名称和对应值写入文件中。

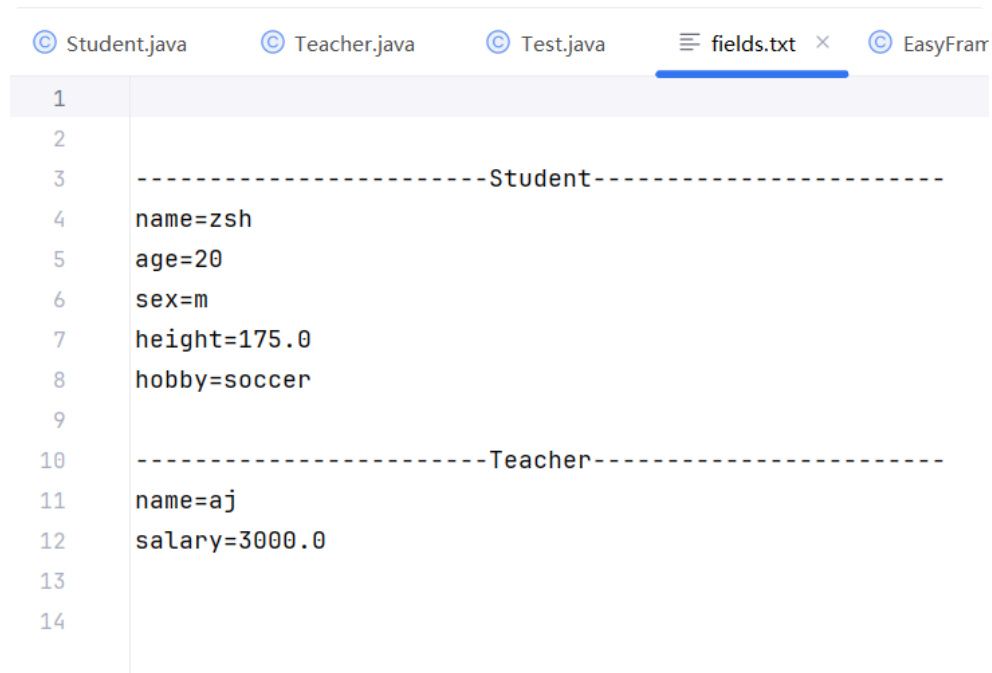
2.4 实现代码

```
1 package framework;
2
3 import java.io.FileOutputStream;
4 import java.io.IOException;
5 import java.io.PrintStream;
6 import java.util.Arrays;
7
8 public class EasyFrameWork {
9     public static void saveObject(Object obj) throws IOException {
10         Class c = obj.getClass();
11
12         try (
13             PrintStream ps = new PrintStream(new
14             FileOutputStream("test11-reflection/src/fields.txt", true));
15         ) {
16             ps.println("-----" + c.getSimpleName() +
17             "-----");
18             Arrays.stream(c.getDeclaredFields()).forEach(field -> {
19                 field.setAccessible(true);
20                 try {
21                     String name = field.getName();
22                     String value = field.get(obj).toString();
23                     ps.println(name + "=" + value);
24                 } catch (IllegalAccessException e) {
25                     throw new RuntimeException(e);
26                 }
27             });
28             ps.println();
29         } catch (IOException e) {
30             throw new IOException();
31         }
```

2.5 测试

```
1 package framework;
2
3 import java.io.IOException;
4
5 public class Test {
6     public static void main(String[] args) throws IOException {
7         Student student = new Student("zsh", 20, 'm', 175.0, "soccer");
8         Teacher teacher = new Teacher("aj", 3000);
9
10        EasyFrameWork.saveObject(student);
11        EasyFrameWork.saveObject(teacher);
12    }
13 }
```

运行上述程序后，可以看到src目录下多出了一个 `fields.txt`，我们点进去看一下：



```
1
2
3 -----Student-----
4 name=zsh
5 age=20
6 sex=m
7 height=175.0
8 hobby=soccer
9
10 -----Teacher-----
11 name=aj
12 salary=3000.0
13
14
```

说明我们制作的简易框架是成功的。

